

Arquitetura e Design Patterns

Importa Aí — Sistema de Gestão e Rastreamento de Encomendas Internacionais

PROJETO

Importa Aí

DISCIPLINA

Design de Software

AUTOR

Higor Paulo Costa

VERSÃO

1.0

DATA

Abril / 2026

CURSO

ADS — 4º Período — PUC Goiás

Arquitetura Hexagonal (Ports & Adapters)

Justificativa da Escolha

- A Arquitetura Hexagonal foi escolhida para o Importa Aí com base nas características concretas do sistema. O projeto possui múltiplos adaptadores de entrada e saída que precisam ser tratados de forma independente do domínio de negócio.
- **Testabilidade:** o domínio não depende de frameworks — pode ser testado em isolamento com JUnit puro, sem necessidade de subir containers ou mocks de infraestrutura.
 - **Independência de mensageria:** RabbitMQ e API REST são adaptadores intercambiáveis. O domínio publica eventos através de uma interface (`EventoPublisher`), sem conhecer o broker.
 - **Projeto solo:** limites claros entre domínio e infraestrutura reduzem acoplamento acidental e facilitam o desenvolvimento incremental.
 - **Adequação ao problema:** o sistema possui 3 adaptadores de entrada (REST, RabbitMQ Consumer, Scheduler) e 5 de saída (MySQL, RabbitMQ Producer, WebSocket, API Correios, API Câmbio).

Estrutura de Camadas

CAMADA	PACOTE	RESPONSABILIDADE
Domínio	<code>domain/</code>	Entidades, Value Objects, Regras de Negócio, Portas (interfaces)
Aplicação	<code>application/</code>	Casos de uso — orquestram o domínio sem conhecer infraestrutura
Infraestrutura	<code>infrastructure/</code>	Adaptadores de entrada (REST, Consumer) e saída (JPA, RabbitMQ, WebSocket)

Regra fundamental: nenhuma importação de `org.springframework` é permitida dentro do pacote `domain/`. O domínio é puro Java.

Design Patterns Aplicados

PATTERN	ONDE É APLICADO	JUSTIFICATIVA
Strategy	Cotação de câmbio (API vs manual vs cache)	Permite trocar a fonte de cotação sem alterar o domínio. Interface <code>CotacaoService</code> com implementações independentes.
Factory Method	Criação de eventos de rastreamento por tipo de etapa	Centraliza a lógica de instanciação de etapas (China, Aeroporto, Correios, etc.) em um único ponto.
Observer (via RabbitMQ)	Publicação de eventos de domínio	Desacopla o produtor dos consumidores. Um evento <code>pedido.criado</code> pode ter N consumidores sem alterar o publicador.
Repository	Acesso a dados (Pedido, Usuário, Notificação)	Abstrai o banco do domínio. O caso de uso depende da interface, não da implementação JPA.

PATTERN	ONDE É APLICADO	JUSTIFICATIVA
Facade	<code>PedidoService</code> como ponto de entrada	Simplifica a interface para o adaptador REST, encapsulando a orquestração do caso de uso.
Chain of Responsibility	Validação de transições de estado do pedido	Cada validador (RN01, RN02) é independente e encadeável, respeitando o princípio Open/Closed.

Princípios SOLID Aplicados

PRINCÍPIO	APLICAÇÃO NO PROJETO
S — Single Responsibility	Cada caso de uso tem uma única responsabilidade (<code>CriarPedidoService</code> , <code>AtualizarStatusService</code>)
O — Open/Closed	Novas fontes de cotação são adicionadas implementando <code>CotacaoService</code> , sem alterar o existente
L — Liskov Substitution	Qualquer implementação de <code>PedidoRepository</code> pode substituir outra sem quebrar o sistema
I — Interface Segregation	Portas de entrada (<code>port/in</code>) e saída (<code>port/out</code>) são interfaces segregadas por responsabilidade
D — Dependency Inversion	O domínio depende de abstrações (interfaces), nunca de implementações concretas de infraestrutura

Diagrama de Pacotes

```

br.com.importaai/
├── domain/
│   ├── model/           # Pedido, EtapaRastreamento, StatusPedido, Notificacao
│   ├── port/
│   │   ├── in/          # CriarPedidoUseCase, AtualizarStatusUseCase, ConsultarPedidoUseCase
│   │   └── out/          # PedidoRepository, EventoPublisher, CotacaoService
│   └── exception/       # TransicaoEstadoInvalidaException, PedidoNaoEncontradoException
├── application/
│   └── usecase/          # CriarPedidoService, AtualizarStatusService, ConsultarPedidoService
├── infrastructure/
│   ├── adapter/
│   │   ├── in/
│   │   │   ├── rest/     # Controllers REST + DTOs
│   │   │   └── messaging/ # Consumers RabbitMQ
│   │   └── out/
│   │       ├── persistence/ # Implementações JPA
│   │       ├── messaging/   # RabbitMQ Producer
│   │       ├── websocket/   # STOMP WebSocket
│   │       └── external/    # API Correios, API Câmbio
│   ├── config/           # RabbitMQConfig, SecurityConfig, WebSocketConfig
│   └── security/         # JwtUtil, JwtAuthFilter

```